

NovaDesk AI - Project Documentation

AI-Powered Customer Support Assistant with Live Response Guidance

Version: 1.0.0

Author: Dhanlaxmi

License: MIT

Date: 2026

Table of Contents

- 1. [Project Overview](#1-project-overview)
 - 2. [Problem Statement](#2-problem-statement)
 - 3. [Proposed Solution](#3-proposed-solution)
 - 4. [System Architecture](#4-system-architecture)
 - 5. [Features](#5-features)
 - 6. [Technology Stack](#6-technology-stack)
 - 7. [Database Design](#7-database-design)
 - 8. [Multi-Agent Pipeline](#8-multi-agent-pipeline)
 - 9. [API Documentation](#9-api-documentation)
 - 10. [Frontend Architecture](#10-frontend-architecture)
 - 11. [Installation & Setup](#11-installation--setup)
 - 12. [Configuration](#12-configuration)
 - 13. [Project Structure](#13-project-structure)
 - 14. [Testing](#14-testing)
 - 15. [Deployment](#15-deployment)
 - 16. [Screenshots & UI Walkthrough](#16-screenshots--ui-walkthrough)
 - 17. [Future Scope](#17-future-scope)
 - 18. [References](#18-references)
-

1. Project Overview

NovaDesk AI is a multi-agent, real-time coaching platform that helps customer support agents perform better during live conversations. Every interaction is analyzed on the fly by a LangGraph-powered pipeline - intent, sentiment, escalation risk, knowledge retrieval, and a suggested reply - before the customer even finishes typing.

The system acts as an intelligent coach that sits alongside support agents, providing instant feedback, suggested responses, and escalation alerts during real-time customer interactions.

Key Highlights

- Real-time WebSocket-based coaching during live conversations
- Multi-agent AI pipeline orchestrated via LangGraph

- RAG-powered knowledge base with ChromaDB vector store
- Three interaction modes: Simulator, Manual, and Replay
- Role-based access control (Admin, Trainer, Agent)
- PDF report generation with coaching analytics
- Works offline with rule-based fallback when no LLM key is configured

2. Problem Statement

Customer support agents face several challenges:

- 1. Lack of real-time guidance - Agents receive coaching only after calls end (post-call reviews), which is reactive and slow.
- 2. High cognitive load - Juggling multiple conversations while maintaining quality is exhausting.
- 3. Inconsistent response quality - Without immediate feedback, agents may provide suboptimal replies.
- 4. Delayed escalation - Critical customer issues may not be flagged until it's too late.
- 5. Limited practice opportunities - New agents lack safe environments to practice before handling real customers.

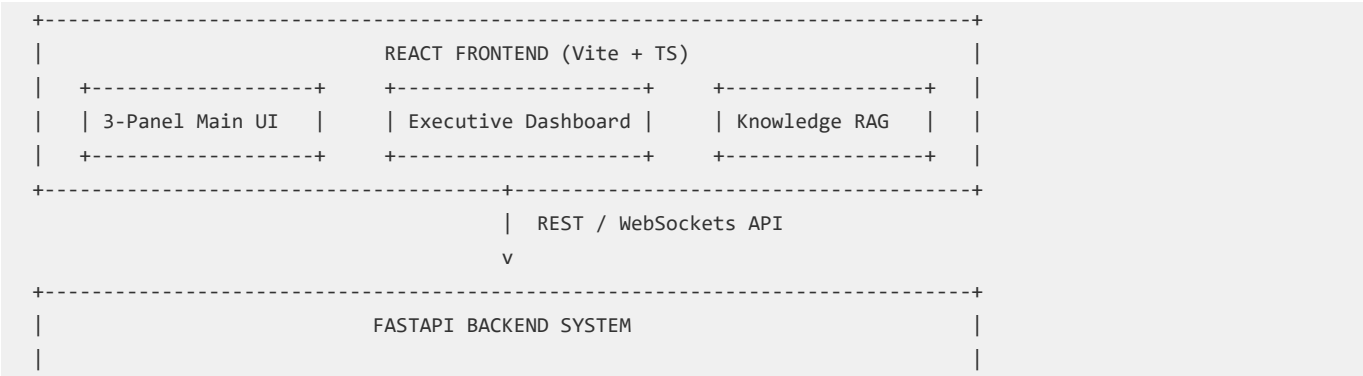
3. Proposed Solution

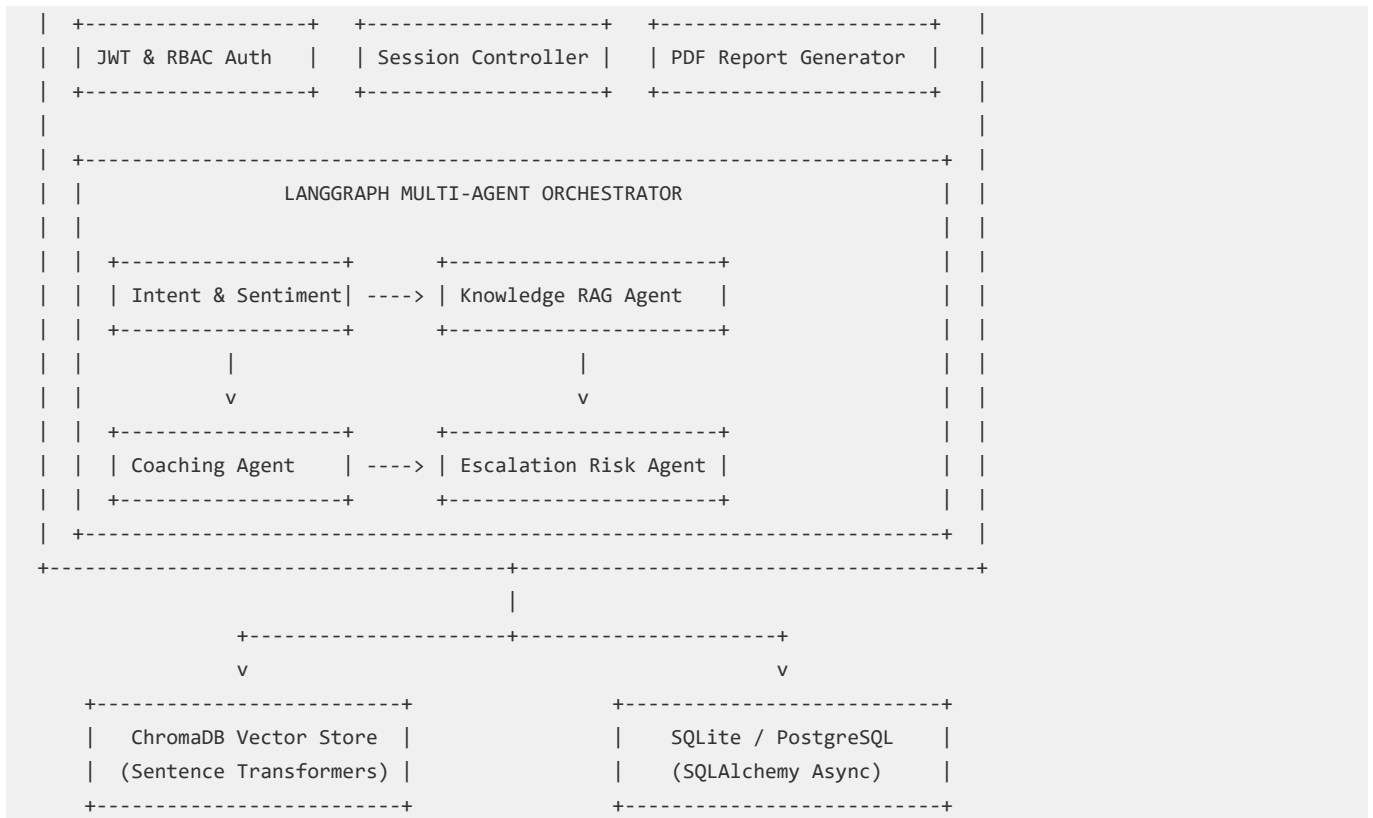
NovaDesk AI addresses these challenges through:

- Live coaching engine - A multi-agent AI pipeline that analyzes every conversation turn in real-time, providing instant scores (tone, grammar, empathy) and suggested replies.
- Customer simulator - AI-powered customer personas that generate realistic conversations for agent training.
- RAG knowledge base - Upload and ingest support documents (PDF, DOCX, TXT) into a ChromaDB vector store for contextual knowledge retrieval during conversations.
- Escalation monitoring - Automatic detection of escalation risk levels (Low, Medium, High, Critical) with recommended supervisor actions.
- Post-interaction analytics - Detailed coaching reports with sentiment journeys, score trends, and downloadable PDF summaries.

4. System Architecture

High-Level Architecture





Data Flow

- 1. User sends message → Frontend via REST API or WebSocket
- 2. Orchestrator receives request → Routes through LangGraph pipeline
- 3. Knowledge RAG Agent → Queries ChromaDB for relevant support articles
- 4. Combined Analysis Agent → Single LLM call returns intent, sentiment, coaching scores, escalation risk, and suggested reply
- 5. Response persisted → Stored in SQLite via SQLAlchemy
- 6. Coaching feedback sent → Returned to frontend in real-time

5. Features

5.1 Live Coaching (WebSocket)

- Real-time conversation streaming with live agent guidance over WebSocket
- Two-phase analysis: instant rule-based provisional result (< 100ms), followed by LLM refinement in background
- Per-turn scores for tone, grammar, empathy, confidence
- Ready-to-send suggested reply with reasoning and improvement tips

5.2 Multi-Agent Pipeline

Agent	Responsibility
Customer Simulator Agent	Generates persona-consistent customer turns (Friendly, Confused,
Intent & Sentiment Agent	Classifies intent, emotion, sentiment, urgency, and frustration s
Combined Analysis Agent	Single-pass intent + sentiment + coaching + escalation analysis (

Knowledge Recommendation Agent	RAG retrieval over ChromaDB with cosine similarity search
Coaching & Response Agent	Scores drafts and generates the ideal reply
Escalation Risk Monitor	Flags Low / Medium / High / Critical risk with recommended action
Post-Interaction Summary Agent	Generates structured coaching summaries and PDF reports

5.3 Interaction Modes

Mode	Description
Simulator Mode	Practice against 6 realistic AI customer personas with emotional
Manual Mode	Paste live customer messages and receive immediate coaching
Replay Mode	Analyze past transcripts turn by turn with full coaching insights

5.4 Knowledge Base Management

- Upload PDF, DOCX, TXT, MD documents
- Auto-chunked (800 chars, 150 overlap) and embedded into ChromaDB
- Semantic search with cosine similarity
- Document versioning and deletion
- Auto-seeding from data/knowledge_base/ on first startup

5.5 Analytics & Reporting

- Executive metrics dashboard with trend charts
- Sentiment journey visualization per session
- Score trends over time
- Downloadable PDF coaching reports
- Cross-session performance analytics

5.6 Accessibility

- Multi-language translation (Hindi, Hinglish, Kannada, Telugu, Tamil, Marathi, Bengali)
- Read-aloud functionality for suggested replies
- Dark/Light theme support

6. Technology Stack

Backend

Layer	Technology	Purpose
API Framework	FastAPI + Uvicorn	Async REST API and WebSocket server
Data Validation	Pydantic v2	Request/response schemas
Orchestration	LangGraph	Multi-agent workflow graph
LLM Integration	OpenAI / Google Gemini / Groq	AI model inference
Vector Store	ChromaDB	RAG document storage and retrieval
Database	SQLAlchemy 2.0 + aiosqlite	Async SQLite ORM
Authentication	JWT (python-jose) + passlib/bcrypt	Secure auth with RBAC
Reports	ReportLab	PDF generation

Real-time	WebSockets (FastAPI)	Live coaching streaming
-----------	----------------------	-------------------------

Frontend

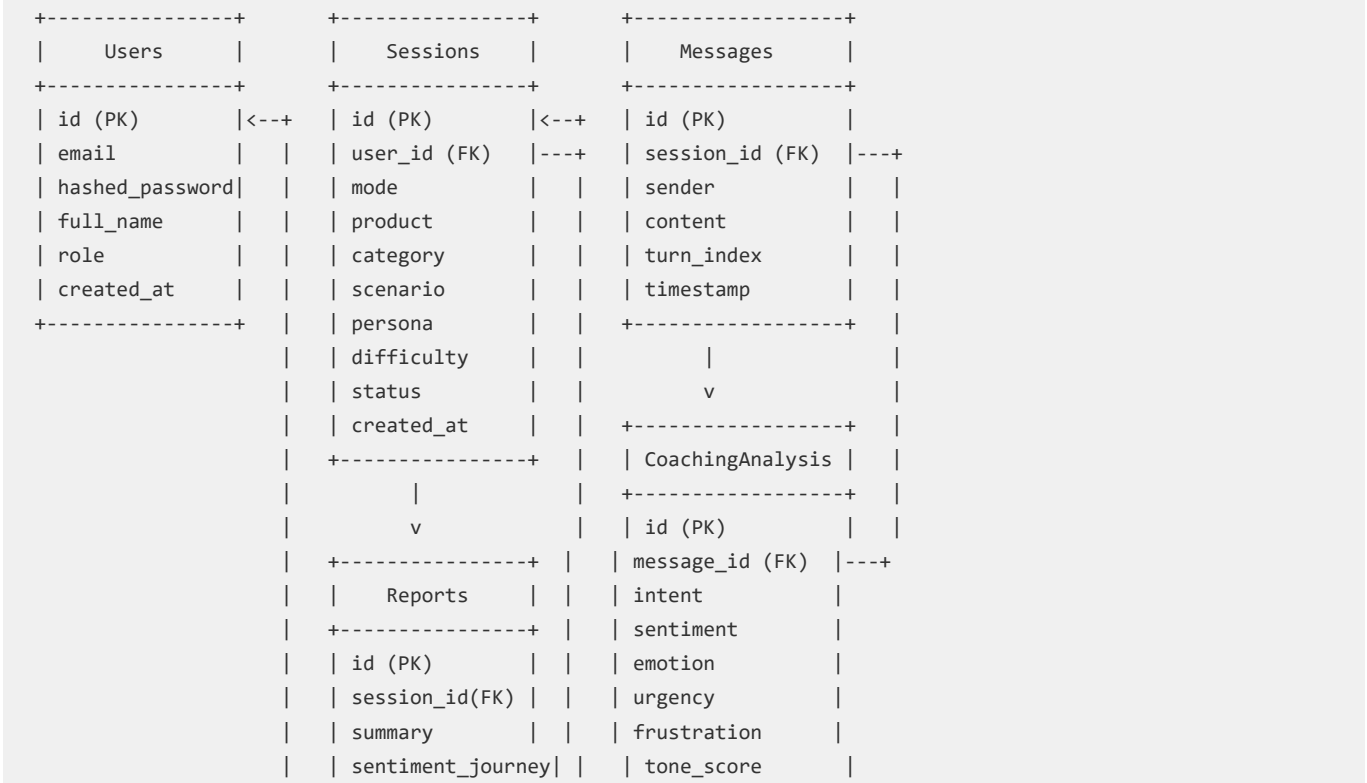
Layer	Technology	Purpose
Framework	React 19 + TypeScript	UI components
Build Tool	Vite 8	Fast dev server and bundler
Styling	Tailwind CSS v4	Glass UI, animations
State Management	Zustand	Global client state
Data Fetching	TanStack Query	Server state caching
Charts	Recharts	Analytics visualizations
Routing	React Router v7	SPA navigation
Motion	Framer Motion	UI animations
HTTP Client	Axios	API communication

DevOps

Tool	Purpose
Docker + Docker Compose	Containerized deployment
Nginx	Reverse proxy for production
Render	Cloud hosting platform

7. Database Design

Entity Relationship Diagram



```

| | resolution_score | | grammar_score |
| | strengths       | | empathy_score  |
| | weaknesses      | | escalation_risk |
| | coaching_tips    | | suggested_reply |
| | pdf_path         | | reasoning       |
| | created_at       | | knowledge_citations |
+-----+ | created_at |
| +-----+
| Documents | | +-----+
+-----+ | | CoachingFeedback |
| id (PK)    | | +-----+
| title      | | | id (PK)          |
| file_path  | | | analysis_id (FK) |
| file_type  | | | user_id          |
| category   | | | rating           |
| chunk_count | | | created_at       |
| created_at | | +-----+
+-----+

+-----+
| SystemSetting |
+-----+
| id (PK)        |
| user_id        |
| key            |
| value          |
+-----+

```

Table Descriptions

Table	Description
users	Stores user accounts with roles (admin, trainer, agent)
sessions	Coaching sessions with mode, persona, scenario, and status
messages	Individual conversation messages within sessions
coaching_analysis	Per-turn AI analysis results (intent, sentiment, scores, suggeste
coaching_feedback	User feedback on coaching suggestions (helpful/not helpful)
documents	Uploaded knowledge base documents metadata
reports	Post-interaction coaching summaries and analytics
system_settings	Application configuration key-value pairs

8. Multi-Agent Pipeline

LangGraph Workflow

```

# Simplified orchestration flow
knowledge_rag → intent_sentiment → END

```

The pipeline consists of two main nodes:

- 1. Knowledge RAG Node - Fast, no LLM call. Retrieves relevant support articles from ChromaDB.

- 2. Combined Analysis Node - Single LLM call that returns all analysis in one round-trip.

Combined Analysis Agent Output

```
{
  "intent": "Technical Issue",
  "sentiment": "Negative",
  "emotion": "Frustrated / Angry",
  "urgency": "High",
  "frustration": 0.75,
  "confidence_score": 0.89,
  "tone_score": 82.5,
  "grammar_score": 91.0,
  "empathy_score": 78.0,
  "suggested_reply": "I understand your frustration with this technical issue. Let me escalate this to our Tier 2 specialist",
  "reasoning": "Acknowledges customer frustration while providing clear next steps.",
  "improvement_tips": [
    "Lead with empathy before technical details",
    "Provide a specific timeline for resolution",
    "Offer a callback if issue persists"
  ],
  "escalation_risk": "High",
  "escalation_reason": "High customer frustration level and urgent dissatisfaction detected.",
  "recommended_action": "Transfer ticket to Tier 2 Engineering Specialist.",
  "knowledge_citations": [...]
}
```

Escalation Risk Levels

Level	Condition	Action
Low	Normal conversation, positive/neutr	Standard support dialogue
Medium	Mild frustration or confusion detec	Provide empathetic reassurance
High	High frustration + urgent dissatisf	Specialist transfer or manager notifi
Critical	Supervisor requested, legal mention	Immediate manager escalation

Fallback Mechanism

When no LLM API key is configured, the system falls back to deterministic rule-based logic:

- Keyword matching for intent classification
- Sentiment analysis via negative/urgent/stress keyword counting
- Pre-defined suggested replies per intent category
- Threshold-based escalation scoring

9. API Documentation

Authentication (/api/v1/auth)

Method	Endpoint	Description	Auth Required
POST	/auth/register	Register new user (admin, trai	No
POST	/auth/login	Authenticate and receive JWT t	No

POST	/auth/forgot-password	Send password recovery instruc	No
GET	/auth/me	Get current user profile and r	Yes

Session Management (/api/v1/session)

Method	Endpoint	Description	Auth Required
POST	/session/create	Create coaching session	Yes
GET	/session/list	List user's coaching sessions	Yes
GET	/session/{id}	Get session messages and histo	Yes
DELETE	/session/{id}	Delete coaching session	Yes

Chat & Live Coaching (/api/v1/chat)

Method	Endpoint	Description	Auth Required
POST	/chat/message	Process chat turn through mult	Yes
POST	/chat/simulator-next	Generate next simulator person	Yes
POST	/chat/manual	Manual mode coaching evaluatio	Yes
POST	/chat/replay	Replay pre-loaded transcript	Yes
POST	/chat/translate	Translate conversation message	Yes
GET	/chat/transcripts	List available transcript file	No
GET	/chat/transcripts/{id}	Get transcript content	No
WS	/chat/ws/{session_id}	Real-time WebSocket coaching s	Yes (JWT)

RAG Knowledge Base (/api/v1/rag)

Method	Endpoint	Description	Auth Required
POST	/rag/upload	Upload document (PDF, DOCX, TX	Yes
POST	/rag/search	Semantic search over knowledge	Yes
GET	/rag/documents	List ingested documents	Yes
DELETE	/rag/documents/{id}	Remove document from vector in	Yes

Reports (/api/v1/report)

Method	Endpoint	Description	Auth Required
POST	/report/generate/{session_i	Generate post-interaction summ	Yes
GET	/report/{session_id}	Get report summary	Yes
GET	/report/{session_id}/pdf	Download PDF report file	Yes

Analytics (/api/v1/analytics)

Method	Endpoint	Description	Auth Required
GET	/analytics/summary	Executive metrics and trend da	Yes

WebSocket Protocol

Connection: ws://localhost:8000/api/v1/chat/ws/{session_id}?token=

Client → Server Messages:


```
{
  "action": "send_message",
  "sender": "customer",
  "content": "I need help with my billing issue",
  "agent_draft": ""
}

{
  "action": "simulator_next"
}
```

Server → Client Messages:

```
{
  "event": "turn_complete",
  "status": "success",
  "message": { "id": "...", "sender": "customer", "content": "...", "turn_index": 1 },
  "coaching": { "intent": "...", "suggested_reply": "...", ... }
}

{
  "event": "coaching_refine",
  "analysis_id": "...",
  "coaching": { ... }
}
```

10. Frontend Architecture

Page Structure

Page	Route	Description
Landing Page	/	Public landing with multi-agent demo
Login	/login	User authentication
Register	/register	New account creation
Main Console	/console	3-panel coaching interface (conversat
Dashboard	/dashboard	Executive overview with key metrics
Sessions	/sessions	Coaching session history and manageme
Knowledge Base	/knowledge	Document upload and management
Analytics	/analytics	Charts and performance trends
Reports	/reports	Post-interaction coaching reports
Settings	/settings	Application configuration
Profile	/profile	User profile management

Component Architecture

```
App.tsx
├─ QueryClientProvider (TanStack Query)
├─ BrowserRouter
│   └─ LandingPage (public)
│   └─ RegisterPage (public)
```

```
| └─ DashboardLayout (protected)
|   └─ Sidebar
|   └─ MobileNav
|   └─ Header
|   └─ PageBackground
|   └─ [Page Content via Outlet]
|       └─ MainConsolePage
|           └─ Conversation Panel
|           └─ Coaching Panel
|               └─ Knowledge Panel
|       └─ DashboardPage
|       └─ SessionsPage
|       └─ KnowledgeBasePage
|       └─ AnalyticsPage
|       └─ ReportsPage
|       └─ SettingsPage
|       └─ ProfilePage
```

State Management (Zustand)

The global store manages:

- user - Current authenticated user
- theme - Dark/Light mode
- sessions - Active coaching sessions
- messages - Current conversation messages
- coaching - Real-time coaching analysis data

Real-Time Communication

The useCoachSocket custom hook manages WebSocket connections:

- Auto-connects on session start
- Sends customer/agent messages
- Receives instant coaching analysis
- Handles background LLM refinement updates
- Graceful reconnection on disconnect

11. Installation & Setup

Prerequisites

- Python 3.11+
- Node.js 18+ and npm
- LLM API key (optional) - OpenAI, Google Gemini, or Groq

Option 1: Local Development

Backend Setup

```
cd customer-support-coach
```

```
# Create and activate virtual environment
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment
copy .env.example .env # Windows
# cp .env.example .env # macOS / Linux
# Add your OPENAI_API_KEY, GEMINI_API_KEY, or GROQ_API_KEY
```

Start Backend

```
uvicorn backend.main:app --reload --port 8000
```

On startup the server automatically:

- Creates the SQLite database (sql_app.db)
- Seeds default demo users (admin, trainer, agent)
- Auto-seeds the knowledge base into ChromaDB

Interactive API docs: <http://localhost:8000/docs>

Frontend Setup

```
cd frontend
npm install
# Configure frontend/.env
npm run dev
```

Open <http://localhost:5173> in your browser.

Option 2: Docker

```
docker-compose -f docker/docker-compose.yml up --build
```

- Frontend: <http://localhost:80>
- Backend API Docs: <http://localhost:8000/docs>

Option 3: Streamlit UI (Legacy)

```
python run.py
```

Opens Streamlit interface at <http://localhost:8501>

12. Configuration

Backend Environment Variables (backend/.env)

Variable	Default	Description
PROJECT_NAME	NovaDesk AI...	API title

PORT	8000	Backend port
DATABASE_URL	sqlite+aiosqlite:///./sql_app.db	Database connection string
SECRET_KEY	*(change in prod)*	JWT signing secret
ACCESS_TOKEN_EXPIRE_MI	1440	Token expiry (24 hours)
OPENAI_API_KEY	*(empty)*	OpenAI API key
GEMINI_API_KEY	*(empty)*	Google Gemini API key
GROQ_API_KEY	*(empty)*	Groq API key
LLM_MODEL	gpt-4o	Default LLM model
GROQ_MODEL	openai/gpt-oss-120b	Groq model name
TEMPERATURE	0.7	LLM temperature
EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6	RAG embedding model
CHROMA_PERSIST_DIRECTO	./chroma_db	Vector store directory
CHUNK_SIZE	800	Document chunk size
CHUNK_OVERLAP	150	Chunk overlap size
UPLOAD_DIR	./uploads	Document upload directory
REPORTS_DIR	./generated_reports	PDF report output directory

Frontend Environment Variables (frontend/.env)

Variable	Default	Description
VITE_API_URL	http://localhost:8000/api/v1	Backend REST base URL
VITE_WS_URL	ws://localhost:8000	Backend WebSocket base URL

13. Project Structure

```
customer-support-coach/
├── backend/
│   ├── agents/
│   │   ├── orchestrator.py      # Multi-agent implementation
│   │   ├── combined_agent.py    # LangGraph workflow orchestrator
│   │   ├── coaching_agent.py    # Single-pass intent+sentiment+coaching
│   │   ├── knowledge_agent.py   # Response scoring & suggestions
│   │   ├── simulator_agent.py   # RAG retrieval agent
│   │   ├── intent_sentiment_agent.py # Customer persona generator
│   │   ├── escalation_agent.py  # Intent & sentiment classifier
│   │   └── summary_agent.py     # Escalation risk monitor
│   │   └── summary_agent.py     # Post-interaction summary
│   ├── api/
│   │   ├── auth.py             # FastAPI route handlers
│   │   ├── session.py          # Authentication endpoints
│   │   ├── chat.py             # Session management
│   │   ├── rag.py              # Chat & WebSocket endpoints
│   │   ├── report.py           # Knowledge base endpoints
│   │   ├── analytics.py        # Report generation
│   │   └── settings.py         # Dashboard metrics
│   ├── authentication/
│   │   ├── jwt.py              # App settings
│   │   ├── rbac.py            # JWT + RBAC
│   │   └── passlib_utils.py    # Token creation/verification
│   └── passlib_utils.py        # Role-based access control
│                               # Password hashing
```

```

|   |   | core/                # Core services
|   |   |   | llm.py           # LLM provider abstraction
|   |   |   | llm_service.py   # LLM service layer
|   |   |   | └ runtime_settings.py # Dynamic settings
|   |   | database/           # Data layer
|   |   |   | models.py        # SQLAlchemy ORM models
|   |   |   | connection.py    # DB connection & session
|   |   |   | └ repository.py  # Data access repository
|   |   | rag/                # RAG pipeline
|   |   |   | vectorstore.py   # ChromaDB vector store
|   |   |   | chunker.py       # Text chunking
|   |   |   | └ ingester.py    # Document ingestion
|   |   | reports/            # PDF generation
|   |   |   | └ pdf_generator.py # ReportLab PDF builder
|   |   | analytics/          # Metrics computation
|   |   |   | └ metrics.py     # Analytics calculations
|   |   | config.py           # Pydantic settings
|   |   | main.py             # FastAPI app entry point
|   |   | └ requirements.txt   # Backend Python dependencies
|   | frontend/
|   |   | └ src/
|   |   |   | └ pages/        # React page components
|   |   |   |   | └ LandingPage.tsx
|   |   |   |   | └ LoginPage.tsx
|   |   |   |   | └ RegisterPage.tsx
|   |   |   |   | └ MainConsolePage.tsx
|   |   |   |   | └ DashboardPage.tsx
|   |   |   |   | └ SessionsPage.tsx
|   |   |   |   | └ KnowledgeBasePage.tsx
|   |   |   |   | └ AnalyticsPage.tsx
|   |   |   |   | └ ReportsPage.tsx
|   |   |   |   | └ SettingsPage.tsx
|   |   |   |   | └ ProfilePage.tsx
|   |   |   | └ components/   # Shared UI components
|   |   |   |   | └ Sidebar.tsx
|   |   |   |   | └ Header.tsx
|   |   |   |   | └ MobileNav.tsx
|   |   |   |   | └ KnowledgeModal.tsx
|   |   |   |   | └ TranscriptPicker.tsx
|   |   |   |   | └ PageBackground.tsx
|   |   |   | └ services/
|   |   |   |   | └ api.ts     # Axios API client
|   |   |   | store/
|   |   |   |   | └ useStore.ts # Zustand global state
|   |   |   | hooks/
|   |   |   |   | └ useCoachSocket.ts # WebSocket hook
|   |   |   | types/
|   |   |   |   | └ index.ts   # TypeScript type definitions
|   |   |   | utils/
|   |   |   |   | └ speech.ts  # Text-to-speech utility
|   |   |   | App.tsx         # Root component with routing
|   |   |   | └ main.tsx      # Entry point
|   |   | system_prompts/     # Agent system prompts (reference)
|   |   |   | └ combined_agent.py
|   |   |   | └ coaching_agent.py
|   |   |   | └ simulator_agent.py

```

```

|   |─ intent_sentiment_agent.py
|   |─ escalation_agent.py
|   └─ summary_agent.py
|─ tests/                                # Backend test suite
|   |─ test_backend.py
|   |─ test_agents.py
|   |─ test_rag.py
|   |─ test_backend_rag.py
|   └─ test_websocket_chat.py
|─ data/
|   |─ knowledge_base/                  # Seed documents for RAG
|   |─ vector_store/                   # Persisted ChromaDB data
|   |─ reports/                         # Generated JSON reports
|   └─ transcripts/                    # Replay mode transcripts
|─ docker/
|   |─ docker-compose.yml
|   |─ Dockerfile.backend
|   |─ Dockerfile.frontend
|   └─ nginx.conf
|─ docs/                                # Project documentation
|   |─ ARCHITECTURE.md
|   |─ API_DOCS.md
|   |─ INSTALLATION.md
|   └─ PROJECT_DOCUMENTATION.md
|─ generated_reports/                  # Generated PDF reports
|─ uploads/                            # Uploaded KB documents
|─ .env                                # Environment configuration
|─ .gitignore
|─ Dockerfile                          # Single-container Dockerfile
|─ LICENSE                             # MIT License
|─ README.md                           # Project README
|─ render.yaml                         # Render deployment config
|─ requirements.txt                     # Root Python dependencies
|─ run.py                              # Streamlit launcher
|─ runtime.txt                          # Python runtime version
|─ AGILE_PLAN.md                       # Agile development plan
|─ sql_app.db                          # SQLite database file

```

14. Testing

Running Tests

```

# From project root
cd customer-support-coach
pytest

# Run specific test file
pytest tests/test_backend.py
pytest tests/test_agents.py
pytest tests/test_rag.py
pytest tests/test_websocket_chat.py

```

Test Coverage

Test File	Coverage Area
test_backend.py	API endpoint tests (auth, session, chat, rag, report)
test_agents.py	Agent logic tests (combined, coaching, simulator, escalation)
test_rag.py	RAG pipeline tests (chunking, ingestion, vector search)
test_backend_rag.py	Integration tests for RAG + API
test_websocket_chat.py	WebSocket real-time coaching tests

Demo Accounts (Auto-Seeded)

Role	Email	Password
Agent	agent@coach.ai	Agent@123456
Trainer	trainer@coach.ai	Trainer@123456
Admin	admin@coach.ai	Admin@123456

15. Deployment

Docker Deployment

```
# Build and run all services
docker-compose -f docker/docker-compose.yml up --build

# Services:
# - Backend (FastAPI): http://localhost:8000
# - Frontend (React): http://localhost:80
# - Nginx: http://localhost:80 (reverse proxy)
```

Render Cloud Deployment

The project includes render.yaml for one-click deployment:

```
services:
  - type: web
    name: novadesk-backend
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn backend.main:app --host 0.0.0.0 --port $PORT
```

Production Checklist

- ☐ Change SECRET_KEY to a secure random value
- ☐ Set proper CORS origins (replace * with domain)
- ☐ Use PostgreSQL instead of SQLite
- ☐ Configure LLM API keys
- ☐ Set up SSL/TLS certificates
- ☐ Configure proper logging
- ☐ Set up database backups
- ☐ Monitor WebSocket connections

16. Screenshots & UI Walkthrough

Landing Page

The public landing page showcases a multi-agent demo animation, demonstrating how tasks route between specialized AI agents.

Main Console (3-Panel Layout)

- Left Panel: Real-time conversation between customer and agent
- Center Panel: Live coaching scores, suggested reply, improvement tips
- Right Panel: Knowledge base citations and relevant articles

Dashboard

Executive overview with key metrics: total sessions, average scores, escalation trends, and agent performance comparisons.

Knowledge Base

Document upload interface supporting PDF, DOCX, TXT, and MD files with auto-ingestion into ChromaDB.

Analytics

Interactive charts showing sentiment journeys, score trends over time, and escalation pattern analysis.

17. Future Scope

- 1. Fine-tuned Models - Train domain-specific models on historical support data for better accuracy
 - 2. Multi-language Support - Real-time coaching in multiple languages
 - 3. Browser Extension - Chrome extension for live coaching on any support platform
 - 4. Integration APIs - Connect with Zendesk, Freshdesk, Intercom, Salesforce
 - 5. Voice Coaching - Real-time audio analysis and voice tone coaching
 - 6. Team Analytics - Manager dashboards with team performance comparisons
 - 7. Gamification - Achievement badges, leaderboards, and training milestones
 - 8. Advanced RAG - Hybrid search (keyword + semantic), query expansion, reranking
 - 9. A/B Testing - Compare coaching strategies across agent groups
 - 10. Mobile App - Native iOS/Android app for coaching on the go
-

18. References

- [FastAPI Documentation](<https://fastapi.tiangolo.com/>)
 - [LangGraph Documentation](<https://langchain-ai.github.io/langgraph/>)
 - [ChromaDB Documentation](<https://docs.trychroma.com/>)
 - [React Documentation](<https://react.dev/>)
 - [Tailwind CSS](<https://tailwindcss.com/>)
 - [SQLAlchemy 2.0](<https://docs.sqlalchemy.org/>)
 - [ReportLab PDF](<https://www.reportlab.com/>)
-

Built with FastAPI, LangGraph, ChromaDB RAG, React, and Tailwind CSS.

NovaDesk AI - Coach at every turn.